

UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number : 07

Lab Report Name: Implementation and Comparison of FCFS and SSTF Disk Scheduling Algorithms

Date of Submission : 04.10.2026

Submitted By:

Name : Ejaj Ahmmod Lemon

ID : 23101160

Section: C2

Submitted To:

Atia Rahman Orthi

Lecturer

University of Asia Pacific

1. Problem Statement:

The objective of this laboratory task is to design and implement the **First Come First Serve (FCFS)** and **Shortest Seek Time First (SSTF) Disk Scheduling Algorithms** for managing disk I/O requests efficiently. In FCFS disk scheduling, the disk requests are serviced in the same order in which they arrive, whereas SSTF selects the pending request that requires the **minimum head movement** from the current disk head position.

The implementation should take a sequence of disk requests and an initial disk head position as input and determine the **seek sequence** for both FCFS and SSTF scheduling techniques. After scheduling the requests, the program should calculate the **Total Head Movement** for each algorithm. Finally, the results should be compared to determine which algorithm provides better performance for the given set of disk requests based on the minimum total head movement.

2. Objectives:

The main objectives of this laboratory experiment are:

- To understand the basic working principles of the **First Come First Serve (FCFS)** and **Shortest Seek Time First (SSTF) Disk Scheduling Algorithms**.
- To implement the **FCFS** disk scheduling algorithm by servicing disk requests according to their given request sequence.
- To implement the **SSTF** disk scheduling algorithm by selecting the pending disk request that requires the **minimum head movement** from the current head position.
- To determine the **seek sequence** and calculate the **Total Head Movement** for both FCFS and SSTF algorithms.
- To compare the performance of **FCFS and SSTF** based on their total head movement.
- To analyze the key **advantages and limitations** of both disk scheduling algorithms.
- To gain practical experience in implementing and evaluating **disk scheduling algorithms using programming**.

3. Souce Code Screenshot:

```
print("========== SSTF ==========")
print("Seek Order:", end=" ")

for i in range(len(sstf_order)):
    print(sstf_order[i], end="")

    if i < len(sstf_order) - 1:
        print(" -> ", end="")

print("\nTotal Head Movement =", sstf_movement)



current = head
fcfs_movement = 0
fcfs_order = [head]

for r in requests:
    fcfs_movement += abs(current - r)
    current = r
    fcfs_order.append(r)


print("\n========== FCFS ==========")
print("Seek Order:", end=" ")

for i in range(len(fcfs_order)):
    print(fcfs_order[i], end="")

    if i < len(fcfs_order) - 1:
        print(" -> ", end="")
```

```
requests = [3, 60, 10, 25, 30, 4]
head = 41

current = head
sstf_movement = 0
pending = requests.copy()
sstf_order = [head]

while pending:

    distances = []

    for r in pending:
        distances.append(abs(current - r))

    nearest_index = distances.index(min(distances))
    nearest = pending.pop(nearest_index)

    sstf_movement += abs(current - nearest)
    current = nearest
    sstf_order.append(nearest)


print("========== SSTF ==========")
print("Seek Order:", end=" ")

for i in range(len(sstf_order)):
    print(sstf_order[i], end="")

    if i < len(sstf_order) - 1:
        print(" -> ", end="")
```

```
print("\n========== FCFS ==========")
print("Seek Order:", end=" ")

for i in range(len(fcfs_order)):
    print(fcfs_order[i], end="")

    if i < len(fcfs_order) - 1:
        print(" -> ", end="")

print("\nTotal Head Movement =", fcfs_movement)



print("\n========== COMPARISON ==========")
print("FCFS Total Head Movement =", fcfs_movement)
print("SSTF Total Head Movement =", sstf_movement)

if sstf_movement < fcfs_movement:
    print("\nSSTF is better than FCFS.")
    print("SSTF has less head movement by",
          fcfs_movement - sstf_movement, "cylinders.")

elif fcfs_movement < sstf_movement:
    print("\nFCFS is better than SSTF.")
    print("FCFS has less head movement by",
          sstf_movement - fcfs_movement, "cylinders.")

else:
    print("\nBoth SSTF and FCFS have the same head movement.")
```

Figure 1: Source Code of the Optimized FCFS and SSTF Disk Scheduling Algorithm.

4. Output Screenshot:

```
========== SSTF ==========
Seek Order: 41 -> 30 -> 25 -> 10 -> 4 -> 3 -> 60
Total Head Movement = 95

========== FCFS ==========
Seek Order: 41 -> 3 -> 60 -> 10 -> 25 -> 30 -> 4
Total Head Movement = 191

========== COMPARISON ==========
FCFS Total Head Movement = 191
SSTF Total Head Movement = 95

SSTF is better than FCFS.
SSTF has less head movement by 96 cylinders.
```

Figure 2: Execution Output of the FCFS and SSTF Disk Scheduling Algorithm.

5. Discussion:

The implemented **First Come First Serve (FCFS)** and **Shortest Seek Time First (SSTF) Disk Scheduling Algorithms** provide two different approaches for servicing disk requests. FCFS services the requests in their given order, while SSTF selects the pending request that requires the **minimum head movement** from the current disk head position.

The implementation takes the disk request sequence and the initial head position as input and determines the seek sequence for both algorithms. The total head movement is then calculated by summing the distance travelled by the disk head between consecutive requests.
Several important observations were obtained from the implementation:

- In **FCFS**, the requests are serviced sequentially according to their given order. For the given request sequence, the seek order is **41 → 3 → 60 → 10 → 25 → 30 → 4**, resulting in a **Total Head Movement of 191 cylinders**.

- In **SSTF**, the closest pending request to the current head position is selected at each step. The seek order obtained is **41 → 30 → 25 → 10 → 4 → 3 → 60**, resulting in a **Total Head Movement of 95 cylinders**.

- The SSTF algorithm performs better for this particular dataset because it significantly reduces the total distance travelled by the disk head.

- The difference between the two algorithms is **96 cylinders**, meaning SSTF requires 96 fewer cylinders of head movement than FCFS for the given request sequence.

- **FCFS** is simple and easy to implement because it does not require searching for the nearest request. Its basic request-processing operation has **O(n)** time complexity.

- The straightforward implementation of **SSTF** repeatedly searches the remaining requests to find the closest one, resulting in $\mathbf{O(n^2)}$ time complexity. However, it can provide lower total head movement compared to FCFS.

- FCFS is fair and predictable because every request is serviced according to its arrival/request order. However, it may cause unnecessary head movement when requests are located far apart.

- SSTF generally reduces unnecessary head movement by servicing nearby requests first, but it may cause **starvation** for requests that are located far from the current head position.

Overall, the experiment provided practical understanding of **disk scheduling, seek sequence generation, and total head movement calculation**. Based on the experimental result, **SSTF is more efficient than FCFS for the given dataset**, as it achieves a total head movement of **95 cylinders compared to 191 cylinders for FCFS**.

6. Conclusion:

The implementation of the **FCFS and SSTF Disk Scheduling Algorithms** successfully demonstrated the fundamental concepts of disk scheduling and head movement optimization. The experiment provided practical experience in determining the **seek sequence** and calculating the **Total Head Movement** for both algorithms.

The **FCFS algorithm** is simple, fair, and easy to implement because disk requests are serviced according to their given order. However, it may result in unnecessary head movement when consecutive requests are located far apart. For the given request sequence, FCFS resulted in a **Total Head Movement of 191 cylinders**.

In contrast, the **SSTF algorithm** selects the request closest to the current disk head position, which significantly reduces unnecessary head movement. For the same request sequence, SSTF resulted in a **Total Head Movement of 95 cylinders**. Therefore, SSTF performed better than FCFS by reducing the head movement by **96 cylinders**.

Overall, the comparison demonstrates that **SSTF is more efficient than FCFS for the given dataset in terms of total head movement**, although SSTF has higher computational overhead and may cause starvation for requests located far from the current head position. The experiment enhanced the understanding of **disk scheduling techniques, algorithmic performance, and the practical advantages and limitations of FCFS and SSTF algorithms**.